

□ Voxora End-to-End Billing & Wallet Workflow

This document explains how Voxora's customer subscriptions, hidden prepaid wallet, call limits, and Stripe automation work together from signup to cancellation.

```
graph TD
    subgraph 1. Onboarding
        A[Customer Subscribes on Web] -->|Stripe Checkout| B(Stripe Webhook)
        B -->|Set Tier & Balance| C[(MongoDB: Org/User)]
    end

    subgraph 2. Inbound Calls
        D[Call Placed via Vapi] -->|Webhook Request| E{Check Wallet Balance}
        E -->|Balance <= $0| F[Route to Voicemail]
        E -->|Balance > $0| G[Calculate Dynamic Duration Cap]
        G -->|Return Config| H[Vapi Limits Call]
    end

    subgraph 3. Call Wrap-Up
        I[Call Ends] -->|end-of-call-report| J[Calculate Cost: $0.20/min]
        J -->|Atomic $inc Deduction| C
    end

    subgraph 4. Refills & Upkeep
        K[Monthly Sub Invoice Paid] -->|SET Balance| C
        L[Buy Overage Pack] -->|INCREMENT Balance| C
        M[Subscription Canceled] -->|Park Number for $5/mo| N{7-Day Grace Period}
        N -->|Paid| O[Keep Number]
        N -->|Unpaid| P[API Release Number]
    end
```

□ The Six Stages of the Billing Lifecycle

1. Signup & Wallet Initialization

- **Action:** A client signs up for the **Professional Plan** (\$249/mo) and completes Stripe checkout.
 - **Stripe Webhook Event:** Catch `checkout.session.completed` / `invoice.paid`.
 - **Backend Logic:**
 1. Set the user's `subscriptionTier` to "professional" and `subscriptionStatus` to "active".
 2. **SET** the client's `walletBalance` to **\$100.00** (representing the 500 included minutes at \$0.20/min).
-

2. Live Call Dynamic Duration Capping

- **Action:** A call is placed to the client's AI receptionist number.
 - **Vapi Assistant Request Webhook:** Vapi calls `/api/vapi-webhook` requesting the assistant setup.
 - **Backend Logic:**
 1. Read the client's `walletBalance` from MongoDB.
 2. **If Balance ≤ \$0.00:** Dynamic router returns a static fallback assistant that acts as a simple voicemail system (zero LLM inference cost).
 3. **If Balance > 0.00 ** :**
Calculate the dynamic call duration limit:
$$\text{maxDurationSeconds} = \min\left(1800, \left\lfloor \frac{\text{walletBalance}}{0.20} \right\rfloor \times 60 \right)$$
Example: If the client has \$0.40 left in their wallet, maxDurationSeconds is dynamically locked to 120 seconds (2 minutes). At the 2-minute mark, Vapi kills the call.**
-

3. Wrap-Up & Atomic Wallet Deduction

- **Action:** The call finishes (either customer hung up, objective completed, or duration cap hit).
- **Vapi End-of-Call Webhook:** Vapi fires the `end-of-call-report` event.
- **Backend Logic:**
 1. Parse the call duration, rounding up to the nearest minute.
 2. Multiply by the standard rate of **\$0.20/minute** to calculate the cost.
 3. Perform an atomic update to deduct the cost from the customer's wallet in MongoDB:

```
await db.collection("users").updateOne(
  { _id: userId },
  { $inc: { walletBalance: -costToDeduct } }
);
```

4. Stripe Automated Refills

- **Action:** The monthly billing cycle shifts, and Stripe charges the client's credit card for their subscription renewal.
 - **Stripe Webhook Event:** Catch `invoice.paid`.
 - **Backend Logic:**
 1. Look up the customer's tier in MongoDB.
 2. **SET** the `walletBalance` back to the base plan limit (**\$50.00** for Starter, **\$100.00** for Professional).
 3. Note: Unused minutes do **not** roll over. The balance is reset, securing high margins from under-utilization.
-

5. Overage Refills (Top-Ups)

- **Action:** A client exhausts their monthly minutes on day 15 and goes into voicemail fallback. They purchase a **\$20.00 Overage Credit Pack** (adds 100 minutes at \$0.20/min).
 - **Stripe Webhook Event:** Catch `checkout.session.completed` for Price ID `price_overage_pack`.
 - **Backend Logic:**
 - To prevent overriding any existing base balances, use MongoDB's `$inc` operator to add the credit: typescript

```
await
db.collection("users").updateOne(      { stripeCustomerId:
stripeCustomerId },      { $inc: { walletBalance:
20.00 } }      );
```
-

6. Subscription Cancellations & Number Parking

- **Action:** A client cancels their subscription.
- **Stripe Webhook Event:** Catch `customer.subscription.deleted`.
- **Backend Logic:**
 1. Set the user's `subscriptionTier` to "parking" and `phoneNumberStatus` to "parked".
 2. Keep the phone number active in Vapi, but change the receptionist response to route calls straight to voicemail.
 3. Trigger an automated email giving the client 7 days to subscribe to the **\$5/month Number Parking Plan** to maintain ownership.

4. If unpaid after 7 days, execute the Vapi API release:

```
await vapi.phoneNumbers.delete(vapiPhoneNumberId);
```

This de-provisions the trunk from Vapi, completely removing carrier costs from your developer account.